Agentic AI: Concepts, Core Capabilities, Architectures, and Use Cases

How autonomous AI systems perceive, reason, act, and learn

November 4, 2026

Contents

1 What Is Agentic AI? 2
- 1.1 Definition . . . 2

2 Agentic AI Compared with Traditional AI 2
- 2.1 Example: customer support . . . 2

3 Core Capabilities: Perceive, Reason, Act, and Learn 3
- 3.1 Perceive: understand the situation . . . 3
- 3.2 Reason: decide what should happen . . . 4
- 3.3 Act: make changes or use tools . . . 4
- 3.4 Learn: improve from feedback . . . 5
- 3.5 How the four capabilities work together . . . 5

4 Single-Agent Systems 6
- 4.1 Typical structure . . . 6

5 Multi-Agent Systems 6
- 5.1 Typical structure . . . 6

6 Single-Agent vs. Multi-Agent: Key Differences 6
- 6.1 A simple decision rule . . . 6

7 Use Cases: Comparing Both Architectures on the Same Problems 7
- 7.1 Use case 1: customer-support request . . . 7
- 7.2 Use case 2: research and report preparation . . . 8
- 7.3 Use case 3: software development . . . 8
- 7.4 Use case 4: travel planning . . . 8

1 What Is Agentic AI?

1.1 Definition

Agentic AI refers to AI systems that can work toward a specified goal by selecting and carrying out a sequence of actions, observing the consequences, and adapting their next steps. An AI agent typically receives an objective, uses available information and tools, maintains some representation of task state, and decides what to do next within boundaries set by its designers or users.

A useful high-level loop is:

1. **Receive a goal:** understand the desired outcome and constraints.
2. **Perceive:** gather relevant information from the user, files, applications, sensors, or other sources.
3. **Reason and plan:** interpret the situation, identify sub-tasks, and choose a next action.
4. **Act:** use an approved tool, application, API, or physical interface.
5. **Observe and evaluate:** inspect the result and determine whether the goal has been achieved.
6. **Adapt or stop:** revise the plan, ask for clarification, request approval, or finish.

This is a loop rather than a one-way pipeline. An action can change the environment, and the new environment may require a different decision. An agent should also have a clear stopping rule; otherwise it may repeat actions, spend excessive resources, or continue after the task is already complete.

2 Agentic AI Compared with Traditional AI

The phrase *traditional AI* covers many approaches, including rule-based systems, statistical machine learning, predictive models, and conventional task-specific AI applications. These systems are not necessarily simple or weak; many are highly accurate within a clearly defined task. The contrast below is a general design comparison, not a claim that every traditional system behaves identically.

2.1 Example: customer support

A traditional intent-classification model may identify that a message concerns a delayed delivery and route it to the correct queue. A generative assistant may draft a helpful explanation. An agentic customer-support system could additionally retrieve the order, check the delivery status, compare it with the service policy, prepare a response, and—if

Dimension	Traditional task-focused AI	Agentic AI
Primary orientation	Produces a prediction, classification, recommendation, or response for a defined input.	Pursues a goal through a sequence of decisions and actions.
Workflow	Often follows a fixed or pre-programmed pipeline.	Can select and revise steps based on context and intermediate results.
Interaction	Commonly request–response or batch processing.	Often iterative: act, observe, evaluate, and act again.
Tools and systems	May be embedded in one application or call a fixed service.	Can select among approved tools, APIs, files, and applications.
Handling change	Usually needs explicit reconfiguration when the task or conditions change.	May adapt its plan when new information or unexpected results appear.
Human role	Human commonly interprets the output and performs follow-up work.	Human sets goals and boundaries, reviews important decisions, and handles exceptions.
Typical output	A score, label, forecast, answer, or recommendation.	A completed workflow or a sequence of actions, along with a result or report.
Main risks	Model error, bias, poor data, and misuse of predictions.	Those risks plus tool misuse, cascading errors, excessive autonomy, and unintended side effects.

Table 1: A general comparison between task-focused AI and Agentic AI.

authorized—issue a permitted remedy. The system should ask for human approval when the proposed remedy exceeds its authority or when the evidence is unclear.

The difference is therefore not that one system uses AI and the other does not. It is the degree to which the system can coordinate steps and cause changes in external systems.

3 Core Capabilities: Perceive, Reason, Act, and Learn

The four capabilities below provide a practical way to understand an agent. They are related and often overlap. A reliable implementation needs all four to be supported by suitable data, tools, and safeguards.

3.1 Perceive: understand the situation

Perception is the process of collecting and interpreting information relevant to the goal. Depending on the application, inputs may include:

- Natural-language instructions, questions, and conversation history.
- Documents, spreadsheets, databases, and knowledge bases.

- Application state, such as calendar availability, order status, or project tasks.
- Images, audio, video, or sensor readings in multimodal systems.
- Feedback from previous actions, such as an API response or an error message.

Perception is more than receiving data. The agent must identify which information matters, distinguish facts from assumptions, notice missing or conflicting details, and preserve relevant context. For example, a travel-planning agent should recognize the destination and dates, but also notice that the traveller has not specified a budget or accessibility requirements if those details materially affect the plan.

Common challenges: incomplete or stale data, ambiguous instructions, noisy sensor input, contradictory sources, and prompt-injection attempts embedded in untrusted documents or webpages. A robust system treats retrieved content as data to evaluate, not as authority to override its instructions or permissions.

3.2 Reason: decide what should happen

Reasoning is the process of interpreting the available information, selecting a strategy, and deciding on the next step. It may include:

- **Goal decomposition:** dividing a broad objective into smaller, manageable tasks.
- **Planning:** ordering tasks and identifying dependencies.
- **Constraint checking:** respecting budgets, deadlines, policies, permissions, and user preferences.
- **Decision-making:** choosing between possible actions based on expected usefulness, cost, risk, and available evidence.
- **Recovery:** diagnosing a failed step and deciding whether to retry, choose another route, or ask a person for help.

For example, to prepare a monthly sales report, an agent may plan to retrieve sales data, validate missing values, calculate totals, compare results with the previous month, create charts, and write a summary. If the data source is unavailable, it should not silently invent figures. It should report the problem and either use an approved alternative or ask for assistance.

An LLM can contribute language understanding and flexible reasoning, but reasoning quality is not guaranteed. Important calculations should be performed with reliable code or tools; factual claims should be checked against trusted sources; and high-impact decisions should have appropriate review.

3.3 Act: make changes or use tools

Action is the ability to affect a system or environment. An agent may act by:

- Searching or querying an approved information source.

- Reading or writing a file, spreadsheet, or database record.
- Calling an API, running a calculation, or executing approved code.
- Creating a draft, updating a task, or preparing a calendar invitation.
- Controlling a robot or another physical system, where applicable.

A tool call should be treated as a consequential operation, not merely as text generation. The agent needs to know what the tool can do, what inputs it requires, what permissions apply, and how to interpret the result. Read-only operations are generally lower risk than actions that send messages, spend money, delete records, or change access permissions.

A sound design uses least-privilege access: give the agent only the permissions needed for its task. It should validate important parameters, use confirmation gates for consequential actions, record what it did, and provide a way to recover from errors where possible.

3.4 Learn: improve from feedback

Learning can mean several different things, and it is important not to confuse them:

1. **In-task adaptation:** the agent updates its plan using feedback during the current task. For example, if a search returns no useful results, it tries a better query.
2. **Context or memory updates:** the system stores permitted task facts or user preferences for later use. Such memory needs clear scope, retention rules, and user controls.
3. **Model improvement:** developers use evaluated data, feedback, or training processes to update the underlying model or policies. This does not necessarily happen automatically during a conversation.

A system can be agentic without changing its model weights. It may simply use feedback to choose better actions within a session. Any persistent learning should be governed by privacy, consent, data quality, and audit requirements. Incorrect feedback can teach the wrong lesson, so learning signals should be checked rather than accepted blindly.

3.5 How the four capabilities work together

Consider an agent asked to identify why a software service is unavailable and prepare a status update. It perceives monitoring alerts and recent deployment records; reasons about possible causes and a safe diagnostic plan; acts by querying approved logs or running read-only checks; then learns from the returned evidence by updating its hypothesis. If a proposed fix would restart production infrastructure, the system should stop at an approval boundary unless it has explicitly been authorized to perform that operation.

4 Single-Agent Systems

A **single-agent system** uses one main agent to manage the task. The agent may call many tools and may use internal subroutines, but one central decision-making process coordinates the workflow.

4.1 Typical structure

1. A user or application supplies a goal.
2. The agent interprets the request and creates a plan.
3. It selects and calls the required tools.
4. It checks tool results and updates its plan.
5. It returns the outcome or asks for human input.

5 Multi-Agent Systems

A **multi-agent system** contains two or more agents that cooperate, coordinate, or sometimes independently examine parts of a problem. Each agent may have a specialized role, such as researcher, analyst, planner, reviewer, or executor. A coordinating agent or workflow manager may assign tasks and combine the results.

5.1 Typical structure

A multi-agent workflow often follows this pattern:

1. A coordinator interprets the overall goal and divides it into sub-tasks.
2. Specialist agents receive bounded assignments.
3. Agents perform their work, potentially in parallel, using approved tools.
4. Their findings are returned to the coordinator in a structured form.
5. The coordinator checks consistency, resolves conflicts, and assembles the result.
6. A human reviews consequential decisions or unresolved disagreements.

6 Single-Agent vs. Multi-Agent: Key Differences

6.1 A simple decision rule

Start with a single agent when the workflow is understandable, sequential, and limited in scope. Consider multiple agents when the work can be divided into independent or

Factor	Single-agent	Multi-agent
Decision structure	One central agent manages the workflow.	Multiple agents handle assigned roles, usually with coordination.
Task suitability	Focused tasks with a manageable number of steps.	Broad tasks with separable workstreams or genuinely different specialties.
Parallel work	Usually limited, unless the agent delegates to ordinary software processes.	Natural fit for independent subtasks that can run concurrently.
Coordination	Relatively straightforward.	Requires task assignment, communication, and conflict resolution.
Cost and latency	Often lower and easier to predict.	Can be higher because of multiple model calls and coordination.
Debugging	Fewer interacting components.	More complex: failures can arise between agents as well as within them.
Consistency	One agent can maintain a coherent task state.	Needs shared definitions and a reliable method for merging outputs.

Table 2: Practical differences between single-agent and multi-agent architectures.

specialized subtasks and when the benefits of parallelism or independent review outweigh the coordination cost. A multi-agent architecture should solve a real decomposition problem, not be added merely because it appears more advanced.

7 Use Cases: Comparing Both Architectures on the Same Problems

The following examples deliberately use the same goal for both architectures. This makes it easier to see that the choice is about system design rather than about a different business problem.

7.1 Use case 1: customer-support request

Goal: resolve a customer's question about a delayed order.

Single-agent approach: one support agent reads the message, retrieves the order record, checks delivery information and relevant policy, drafts a response, and offers an authorized remedy. It escalates if the case is ambiguous or outside policy.

Multi-agent approach: a triage agent identifies the issue; an order-status agent retrieves shipment information; a policy agent checks the applicable rules; and a response agent prepares the message. A coordinator reconciles the findings and routes unusual cases to a human.

When to choose: a single agent is usually sufficient for common, well-defined support requests. A multi-agent design may help a large support operation with separate systems, specialist policies, or high volumes, provided the hand-offs are reliable.

7.2 Use case 2: research and report preparation

Goal: prepare a report on a market or technology topic.

Single-agent approach: one research agent identifies questions, searches approved sources, extracts evidence, compares findings, checks citations, and writes the report.

Multi-agent approach: a research coordinator divides the topic into subtopics. Several researcher agents investigate different areas; a fact-checking agent checks claims and source quality; an analyst synthesizes the evidence; and an editor prepares the final report. The coordinator resolves contradictions and ensures the final document distinguishes evidence from interpretation.

When to choose: a single agent works well for a short, focused brief. Multi-agent research can be useful for broad topics with independent research streams, but it needs source tracking and careful synthesis to avoid duplicated or contradictory claims.

7.3 Use case 3: software development

Goal: implement a small feature in an existing application.

Single-agent approach: one coding agent inspects the codebase, identifies relevant files, proposes a change, edits the code, runs tests, and reports what changed. It requests review before merging or deploying if required.

Multi-agent approach: a planning agent translates the feature request into acceptance criteria; a code-analysis agent examines the architecture; an implementation agent makes the change; a test agent develops and runs tests; and a reviewer agent checks correctness and maintainability. A coordinator combines the results.

When to choose: a single agent is appropriate for a small, localized change. Multiple agents may help with larger changes that have separable implementation, testing, and review work. They should not edit the same files concurrently without explicit coordination, and production deployment should remain behind strict approval controls.

7.4 Use case 4: travel planning

Goal: create a feasible trip itinerary within a budget and date range.

Single-agent approach: one travel agent collects preferences, compares transport and accommodation options, checks dates and constraints, drafts an itinerary, and presents trade-offs for approval.

Multi-agent approach: a transport agent compares routes; an accommodation agent researches stays; a local-activities agent finds suitable activities; and a budget agent checks the combined estimate. A coordinator checks that dates, locations, and travel times fit together.

When to choose: a single agent is often best for a short trip. Multiple agents may help when there are many destinations, constraints, or research streams. Any booking or payment should require explicit user authorization and final confirmation of the details.